



Язык программирования Java

№ Фамилия Имя – Группа

Ссылки



План занятий

2 недели назад

- ① Классы и объекты
- ② Методы и конструкторы
- ③ Фундаментальные принципы ООП

Сегодня

- ① Внутренние классы
- ② Наследование

ВНЕШНИЙ

Внутренний класс

Класс, описанный **внутри другого класса**. Класс, в котором объявлен внутренний класс, называется **внешним**

Внутренний класс имеет особенности

- ① Члены внутреннего класса доступны только в его пределах**
- ② Члены внутреннего класса недоступны во внешнем классе**
- ③ Может обращаться к членам внешнего класса напрямую**

Внутренний класс

```
1 class Alpha {
2     int number = 256;
3     void show() {
4         Bravo innerClass = new Bravo();
5         innerClass.display();
6     }
7     // Внутренний класс
8     class Bravo {
9         void display() {
10             System.out.println("Поле number: " + number);
11         }
12     }
13 }
14 class Main {
15     public static void main(String[] args) {
16         Alpha outerClass = new Alpha();
17         outerClass.show(); // Поле number: 256
18     }
19 }
```

Наследование

Наследование

```
1 class Alpha {
2     int number;
3     void show() {
4         System.out.println("Поле number: " + number);
5     }
6 }
7 class Bravo extends Alpha {
8     int value;
9     void display() {
10        System.out.println("Поле value: " + value);
11    }
12    void sum() {
13        System.out.println("Сумма " + (number + value));
14    }
15 }
16 class Main {
17     public static void main(String[] args) {
18        Alpha objA = new Alpha();
19        objA.number = 128;
20        objA.show(); // Поле number: 128
21        Bravo objB = new Bravo();
22        objB.number = 64;
23        objB.value = 36;
24        objB.show(); // Поле number: 64
25        objB.display(); // Поле value: 36
26        objB.sum(); // Сумма: 100
27    }
28 }
```

Наследование и private

```
1 class Alpha {
2     private char symbol;
3     void show() {
4         System.out.println("Символ: " + symbol);
5     }
6     void set(char s) {
7         symbol = s;
8     }
9 }
10
11 class Bravo extends Alpha {}
12
13 class Main {
14     public static void main(String[] args) {
15         Bravo objB = new Bravo();
16         objB.symbol = 'B';
17     }
18 }
```



Ошибка компиляции

The field Alpha.symbol is not visible

Наследование и private

```
1 class Alpha {
2     private char symbol;
3     void show() {
4         System.out.println("Символ: " + symbol);
5     }
6     void set(char s) {
7         symbol = s;
8     }
9 }
10
11 class Bravo extends Alpha {}
12
13 class Main {
14     public static void main(String[] args) {
15         Bravo objB = new Bravo();
16         objB.set('B');
17         objB.show(); // Символ: В
18     }
19 }
```

Пакет

Контейнер для классов, их пространство имен. Все классы разбиваются по группам, которые и называют **пакетами**. Имя класса должно быть уникальным в пределах пакета.

Пакеты

```
1 package ru;  
2  
3 class Main {  
4     public static void main(String[] args) {  
5     }  
6 }
```

Иерархия пакетов строго соответствует файловой структуре и размещению пакетов в ней

Например, инструкция `package java.awt.image` означает, что файлы подпакета `image` размещены в каталоге `java/awt/image`

Обращение к классам, размещенным в пакетах

Если необходимо обратиться к классам, размещенным во внешних пакетах, то необходимо **указать полное имя класса**: через точку перечисляется вся иерархия пакетов, где размещен нужный класс

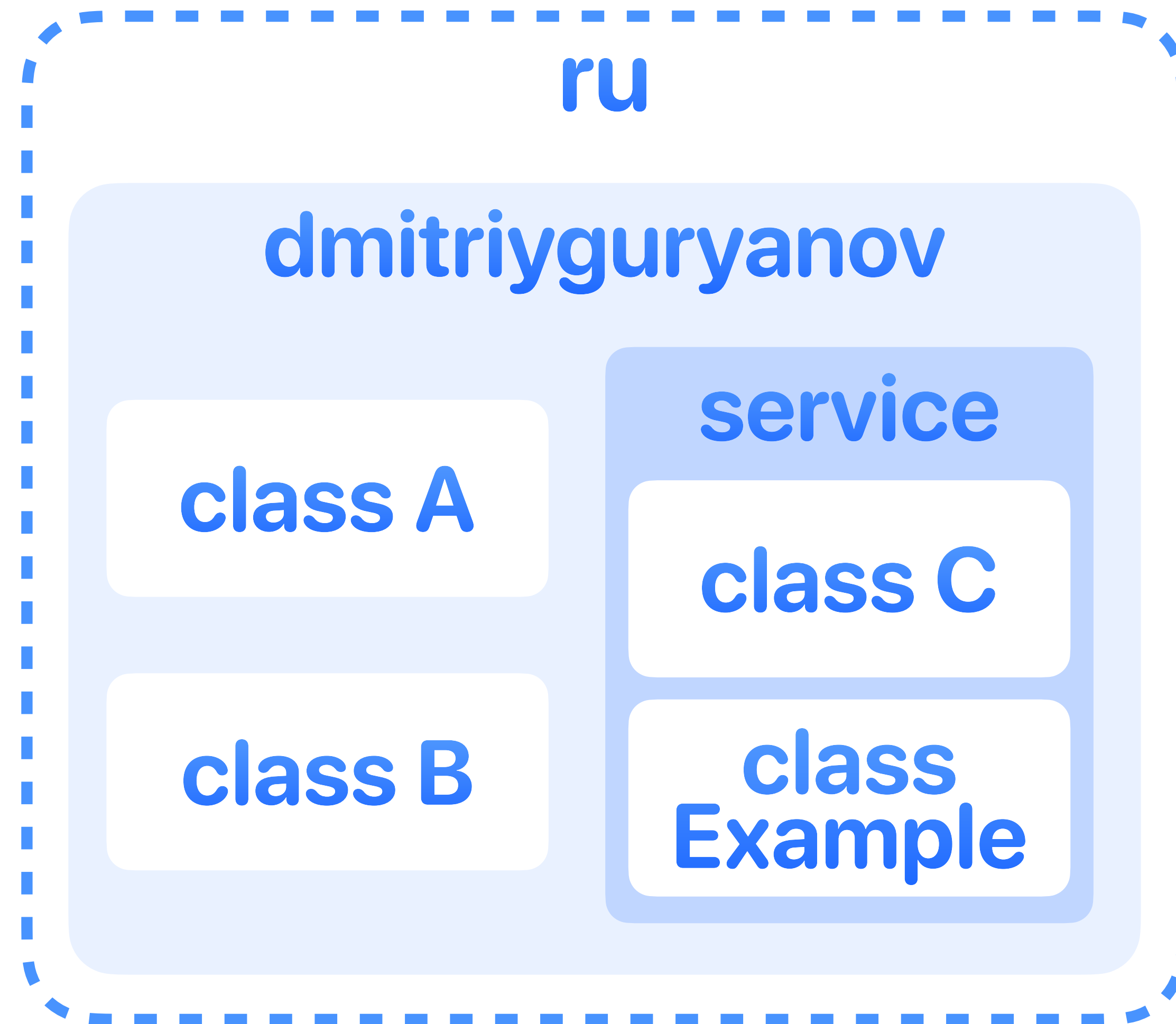
Обращение к классам в пакетах

```
1 package ru;
2 import java.util.Scanner;
3
4 class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7     }
8 }
```

Наименование пакетов

```
1 package ru.psu;  
2 package com.dmitriyguryanov;  
3 package org.example;
```

Обращение к классу

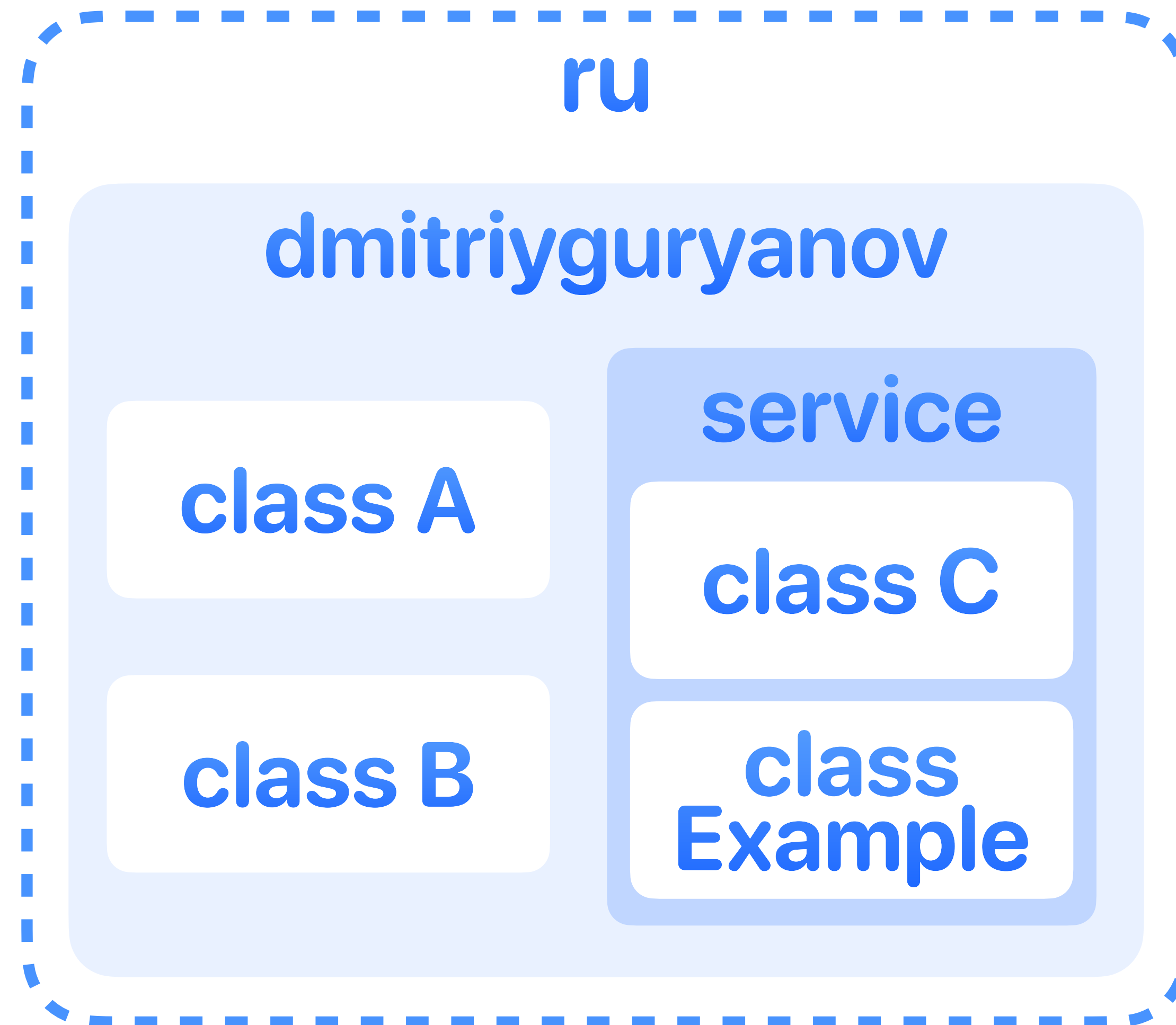


```
import ru.dmitriyguryanov.service.Example
```

Ограничения на импорт пакетов

- ① Импортировать можно только открытые классы
- ② Пакет `java.lang` можно не импортировать
- ③ Открытые классы в пакете должны быть в разных файлах

Модификаторы доступа



Доступность членов класса

- ⊛ Открытые члены (`public`) доступны **везде**
- ⊛ Закрытые члены (`private`) доступны только **в пределах своего класса**
- ⊛ Открытые члены класса без спецификатора доступа доступны **в пределах своего пакета**
- ⊛ Защищенные члены класса (`protected`) доступны **в пределах своего пакета, а также в подклассах**

Конструкторы и наследование

Объект подкласса создается частями: сначала создается та часть, которая определяется суперклассом, а затем достраивается часть, связанная с подклассом

Конструкторы и наследование

```
1 // Конструктор подкласса
2 Bravo (int n, int a) {
3     super(n);
4     this.a = a;
5 }
```

Конструкторы и наследование

```
1 class Alpha {
2     int number;
3     void show() {
4         System.out.println("Поле number: " + number);
5     }
6     Alpha() {
7         this.number = 100;
8         show();
9     }
10    Alpha(int n) {
11        this.number = n;
12        show();
13    }
14 }
15 class Bravo extends Alpha {
16     char symbol;
17     void display() {
18         System.out.println("Поле symbol: " + symbol);
19     }
20    Bravo() {
21        super();
22        this.symbol = 'A';
23        display();
24    }
25    Bravo(int n, char s) {
26        super(n);
27        this.symbol = s;
28        display();
29    }
30 }
```

Конструкторы и наследование

```
1 class Main {  
2     public static void main(String[] args) {  
3         Bravo objB1 = new Bravo(); // Поле number: 100, Поле symbol: A  
4         Bravo objB2 = new Bravo(200, 'B'); // Поле number: 200, Поле symbol: B  
5     }  
6 }
```

Переопределение методов

Если метод унаследован, но его функциональность в силу каких-то причин не устраивает, можно прибегнуть к **переопределению**. В таком случае **переопределяемый метод в подклассе описывается заново**

Переопределение и перегрузка

Между переопределением и перегрузкой методов существует отличие. При **перегрузке** методы имеют **одинаковые названия, но разные аргументы**. При **переопределении** совпадают не только названия, но и **тип результата и аргументы**.

Переопределение методов

```
1 class Alpha {
2     private int number;
3     void show() {
4         System.out.println("Поле number: " + number);
5     }
6     void set(int n) {
7         this.number = n;
8     }
9     Alpha(int n) {
10         set(n);
11     }
12 }
13 class Bravo extends Alpha {
14     private char symbol;
15     @Override
16     void show() {
17         super.show();
18         System.out.println("Поле symbol: " + symbol);
19     }
20     void set(int n, char s) {
21         set(n);
22         this.symbol = s;
23     }
24     Bravo(int n, char s) {
25         super(n);
26         this.symbol = s;
27     }
28 }
```

Замещение полей при наследовании

```
1 class Alpha {
2     int code;
3 }
4 class Bravo extends Alpha {
5     int code;
6     void set(int m, int n) {
7         super.code = m;
8         this.code = n;
9     }
10    void show() {
11        System.out.println("Alpha: " + super.code);
12        System.out.println("Bravo: " + code);
13    }
14 }
15 class Main {
16     public static void main(String[] args) {
17         Bravo obj = new Bravo();
18         obj.set(128, 256);
19         obj.show(); // Alpha: 128 Bravo: 256
20         obj.code = 512;
21         obj.show(); // Alpha: 128 Bravo: 512
22     }
23 }
```

Конструкторы в многоуровневом наследовании

```
1 class Alpha {
2     int alpha;
3     Alpha(int a) {
4         this.alpha = a;
5         System.out.println("alpha: " + alpha);
6     }
7 }
8 class Bravo extends Alpha {
9     int bravo;
10    Bravo(int a, int b) {
11        super(a);
12        this.bravo = b;
13        System.out.println("bravo: " + bravo);
14    }
15 }
16 class Charlie extends Bravo {
17     int charlie;
18    Charlie(int a, int b, int c) {
19        super(a, b);
20        this.charlie = c;
21        System.out.println("charlie: " + charlie);
22    }
23 }
24 class Main {
25     public static void main(String[] args) {
26         new Charlie(1, 2, 3); // alpha: 1, bravo: 2, charlie: 3
27     }
28 }
```

Абстрактные классы

Методы и классы могут быть абстрактными. **Абстрактный метод** – метод, тело которого не описано в классе. Для такого метода объявления только сигнатура. При объявлении метода указывается **abstract**

Класс, который содержит хотя бы один такой метод, тоже называется абстрактным

Абстрактные классы

```
1 abstract class Alpha {
2     abstract void first();
3     void second() {
4         System.out.println("Второй метод");
5     }
6 }
7 class Bravo extends Alpha {
8     void first() {
9         System.out.println("Первый метод");
10    }
11 }
12 class Main {
13     public static void main(String[] args) {
14         Bravo objB = new Bravo();
15         objB.first();
16         objB.second();
17         Alpha objA;
18         objA = objB;
19         objA.first();
20         objA.second();
21     }
22 }
```

Ссылки

